Implementación de Buscaminas en Lenguaje Ensamblador MARIE

Emilio Puga · Eduardo Cedeño · Jorge Marcillo
Organización de Computadores CMP 3004 · Universidad San Francisco de Quito

DESCRIPCIÓN DEL PROYECTO

Implementación funcional del juego Buscaminas sobre el simulador MARIE.js, una arquitectura de 16 bits con instrucciones mínimas, sin soporte nativo de funciones, pila ni estructuras de control.

El tablero de 16×16 es generado externamente en Python y cargado como bloque de memoria. Toda la lógica del juego se implementa íntegramente en lenguaje ensamblador MARIE.

MAPAS EN MEMORIA

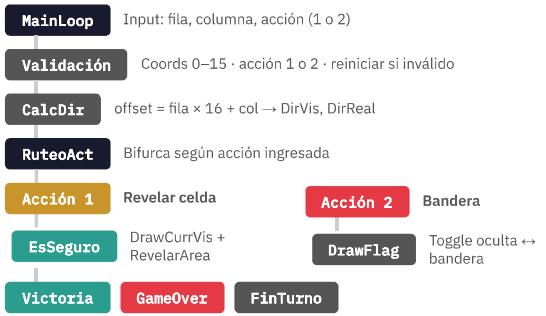
Dirección	Mapa	Contenido
0x100	Real	0–8 minas adyacentes; −1 = mina
0x200	Visible	0 oculta · 1 revelada · 2 bandera
0xF00	Display	Colores RGB565 renderizados

REVVECINO — EXPANSIÓN LOCAL

- **Verificación:** si fue procesada (Visible ≠ 0),no revela. Si es mina (Real = -1), se descarta.
- **Revelar:** marca la celda como visible (1) y la envía a `DrawCurrVis`.
- **Propagación:** decrementa `FaltanRevelar` y activa `HuboCambio` para continuar el flood fill.

```
RevVecino(vecOffset):
   TmpVis = BaseVis + vecOffset
   if visible[TmpVis] != 0: return
   if real[TmpVis] == -1: return
   visible[TmpVis] = 1
   CurrVis = TmpVis
   DrawCurrVis(CurrVis, ValVecino)
   HuboCambio = true
   FaltanRevelar -= 1
```

FLUJO PRINCIPAL DEL JUEGO



FLOOD FILL ITERATIVO — REVELARAREASR

Cuando una celda revelada vale 0, se expanden automáticamente todas las celdas adyacentes vacías y sus perímetros numéricos. Se usan pasadas iterativas en lugar de recursión (MARIE no tiene stack).

```
RevelarArea(valReal):
   if valReal != 0: return
   do:
      HuboCambio = false
      para CurrOffset de 0 a 255:
         if visible[i]!=1 or real[i]!=0: siguiente
         GetFilaCol → filaActual, colActual
         para cada vecino [N,S,E,O,NE,NW,SE,SW]:
         RevVecino(vecOffset)
   while HuboCambio == true
```

- **GetFilaCol:** divide CurrOffset entre 16 por restas sucesivas → fila y columna actuales.
- **RevVecino:** si oculto y no mina → revela, dibuja, decrementa FaltanRevelar, HuboCambio = 1.

REVELAR CELDA — LÓGICA CENTRAL

- **Guardas previas:** si la celda tiene bandera (2) o ya está revelada (1), saltar a FinTurno.
- **LeerReal:** si valor = −1 → **GameOver** (display rojo + Halt).
- **EsSeguro:** escribir 1 en Mapa Visible, llamar a DrawCurrVis, decrementar FaltanRevelar.
- **RevelarAreaSR:** si valor = 0, ejecutar flood fill. Si FaltanRevelar = 0 → **Victoria**.

```
Revelar:
   if estadoVis == 2: goto FinTurno // bandera
   if estadoVis == 1: goto FinTurno // revelada
   if valReal == -1: goto GameOver // mina
   tableroVisible[offset] = 1
   DrawCurrVis(CurrVis, valReal)
   FaltanRevelar -= 1
   RevelarArea(valReal)
```

CONCLUSIONES

- Se implementó un juego interactivo completo sobre una arquitectura de solo 16 instrucciones, demostrando que la separación de responsabilidades es posible incluso en ensamblador básico.
- La división en tres mapas de memoria (Real, Visible, Display) permitió desacoplar la lógica del juego de la visualización, facilitando el trabajo en paralelo entre los miembros del equipo.
- El flood fill iterativo con pasadas múltiples resuelve la expansión de celdas vacías sin necesidad de recursión ni estructuras de datos complejas.
- La generación externa del tablero en Python ilustra un patrón real en sistemas embebidos: preparación de datos fuera de la máquina objetivo.